

LeetCode 2132: Stomping the Grid

Advanced Grid Manipulation via 2D Prefix Sums & 2D Difference Arrays

Author: Arpit Raj | Institution: LNMIIT Jaipur | Date: July 9, 2026 | Target Companies: Rubrik, Google

Problem Overview & Rules

Given an $m \times n$ binary matrix grid where:

- 0 represents an **empty** cell.
- 1 represents a **filled** cell.

We are given a stamp of size $h \times w$. We want to determine if it is possible to fit/place stamps to cover **all empty cells (0s)** under the following rules:

1. Every stamp must be fully contained within the grid boundaries.
2. Stamps **must not touch or cover any filled cells (1s)**.
3. We can use an **unlimited number of stamps**.
4. Stamps can **overlap** with each other freely.
5. **Rotation of the stamp is NOT allowed**.

Visual Example

Input Grid Matrix:

0	0	0
0	1	0
0	0	0

A stamp of size 2×2 can only be placed if every cell covered by that specific placement is completely empty (0).

Algorithmic Strategy

1. Validating Stamp Placements (2D Prefix Sum)

To know if a stamp of size $h \times w$ can be placed with its top-left corner at (r_1, c_1) and bottom-right corner at (r_2, c_2) , we must ensure that the rectangle contains **only 0s**. If the range sum of 1s inside this rectangle is exactly 0, then a stamp can safely be placed there.

Instead of checking every cell inside the rectangle manually (which would take $O(h \times w)$), we build a **2D Prefix Sum Array** to execute each query in $O(1)$ time.

2. Efficient Coverage Tracking (2D Difference Array)

When a stamp placement is found to be valid, we need to mark all cells inside that $h \times w$ rectangle as "covered". A naive approach of looping through every cell inside the stamp would lead to a slow time complexity of $O(m \times n \times h \times w)$.

To optimize this to $O(m \times n)$, we utilize a **2D Difference Array** (named `mark` in our implementation). Instead of adding coverage to each cell individually, we mark only the boundaries/corners of the rectangle to simulate the start and end effects:

Visual Scheme of 2D Difference Array Markings:

For a valid rectangle starting at (r_1, c_1) and ending at (r_2, c_2) :

- `mark[r1][c1] += 1` (Start Effect)
- `mark[r2 + 1][c1] -= 1` (Stop Effect)
- `mark[r1][c2 + 1] -= 1` (Stop Effect)
- `mark[r2 + 1][c2 + 1] += 1` (Counteracting double-subtraction)

Step-by-Step Execution Flow

Step 1: Build Prefix Sum Array

Construct a 2D prefix sum array over the original grid to quickly find the count of 1s in any subgrid.

Step 2: Check Every Valid Stamp Window

Iterate through every possible offset pair $(roff, coff)$ where a stamp can fit. If the sum of cells in that range is 0, update the difference array (`mark`).

Step 3: Recover the True Coverage Matrix

Compute the 2D prefix sum over the `mark` difference array to calculate the actual coverage count for each cell.

Step 4: Final Validation Check

Verify if any empty cell (`grid[i][j] == 0`) has a coverage count of 0. If found, return false. Otherwise, return true.

C++ Source Code Implementation

```
class Solution {
public:
    bool possibleToStamp(vector<vector<int>>& grid, int h, int w) {
        int n=grid.size();
        int m=grid[0].size();

        vector<vector<int>> pref(n,vector<int>(m));
        vector<vector<int>> mark(n+1,vector<int>(m+1));

        for(int i=0;i<n;i++){    //prefic array building
            for(int j=0;j<m;j++){
                pref[i][j]=grid[i][j];

                if(i>0){
                    pref[i][j]+=pref[i-1][j];
                }

                if(j>0){
                    pref[i][j]+=pref[i][j-1];
                }

                if(i>0 && j>0){
                    pref[i][j]-=pref[i-1][j-1];
                }
            }
        }

        for(int roff=0;roff+h<=n;roff++){    //checking all possible stamp positions and mark
            for(int coff=0;coff+w<=m;coff++){
                int r1=roff;
                int c1=coff;
                int r2=roff+h-1;
                int c2=coff+w-1;

                int sum=pref[r2][c2];
                if(r1>0){
                    sum-=pref[r1-1][c2];
                }

                if(c1>0){
                    sum-=pref[r2][c1-1];
                }

                if(r1>0 && c1>0){
                    sum+=pref[r1-1][c1-1];
                }

                if(sum==0){
                    mark[r1][c1]++;
                    mark[r2+1][c1]--;
                    mark[r1][c2+1]--;
                    mark[r2+1][c2+1]++;
                }
            }
        }
    }
};
```

```

    }
}

for(int i=0;i<n;i++){ //building the mark array just like prefix array
    for(int j=0;j<m;j++){
        if(i>0){
            mark[i][j]+=mark[i-1][j];
        }
        if(j>0){
            mark[i][j]+=mark[i][j-1];
        }

        if(i>0 && j>0){
            mark[i][j]-=mark[i-1][j-1];
        }
    }
}

for(int i=0;i<n;i++){ //checking if after everything is any position left left to be
    for(int j=0;j<m;j++){
        if(grid[i][j]==0 && mark[i][j]==0){
            return false;
        }
    }
}

return true;
}
};

```